Computer Networking: Security (CSE 3752)

Experiment 5

Aim:

Implementation of DES (Data Encryption Standard) block cipher for secured transmission of digital information in computer network.

Objectives:

1. Understand and implement of DES round key generation.
2. Execution of DES algorithm for encryption and decryption of digital information.

Exercises:

Using simplified DES find the following for the given 8-bit plaintext 10010111 and 10-bit key 1010000010

1. Find the permuted key using the P10 table given as 3 5 2 7 4 10 1 9 8 6 and the round 1 key (K1)
   using the P8 table 6 3 7 4 8 5 10 9
2. Find the output of initial permutation as L0 and R0 using the given IP table 2 6 3 1 4 8 5 7
3. Find the output of expansion permutation on R0
4. Find the output of XOR (EP(R0),K1)
5. Find the output of the given S-boxes

$$S0 = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 1 & 0 & 3 & 2 \\ 3 & 2 & 1 & 0 \\ 0 & 2 & 1 & 3 \\ 3 & 1 & 3 & 2 \end{bmatrix} \end{array} \qquad S1 = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 3 \\ 3 & 0 & 1 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix} \end{array}$$

6. Find the output of the DES function using the permutation table P4 (2 4 3 1).

DES Round Key Generation

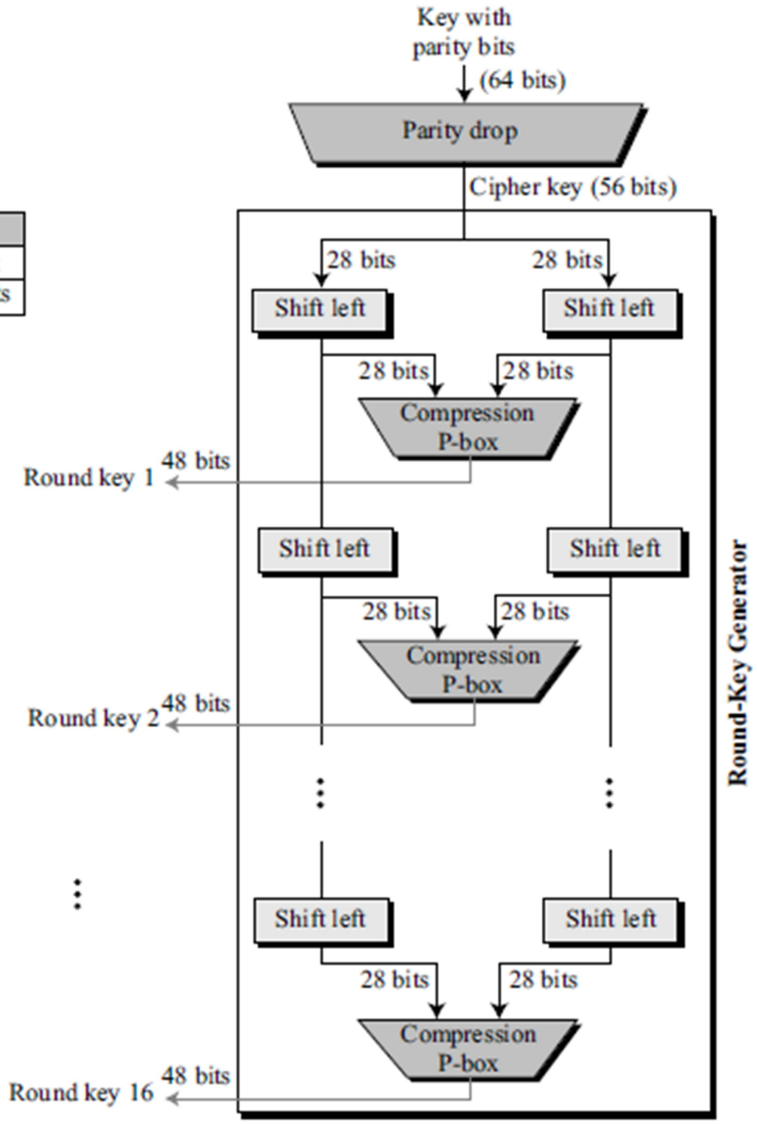
****

Objective-1: Write a Python program to generate 16 round keys (48-bit each) from a 64-bit DES key.

Task 0: Understand the key generation process

Shifting

Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits



Task 1: Define Required Tables

Create the following tables in your code:

- Parity Drop Table
- Shift Left Table
- Compression P-Box Table

Table 6.12 *Parity-bit drop table*

57	49	41	33	25	17	09	01
58	50	42	34	26	18	10	02
59	51	43	35	27	19	11	03
60	52	44	36	63	55	47	39
31	23	15	07	62	54	46	38
30	22	14	06	61	53	45	37
29	21	13	05	28	20	12	04

Table 6.13 *Number of bit shifts*

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bit shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Table 6.14 *Key-compression table*

14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Task 2: Write Format Conversion Functions

Write two methods ***hex_to_bin(str)*** and ***bin_to_hex(str)*** to convert the hexadecimal string to binary and vice versa. Missing bits should be filled with 0s.

Task 3: Write Permutation Function

Write a method ***permute(data, table)*** to rearrange bits by referring a table.

Task 4: Write Left Shift Function

Write a method ***shift_left(data, n)*** to perform circular left shift operation.

Task 5: Write Key Generation Function

Write a method ***generate_round_keys(key_64bit)*** to generate 16 round keys.

- Perform the parity drop operation.
- Divide into two halves: left and right.
- Perform circular left shift for both the halves by referring the shift left table.
- Merge the results of both halves.
- Apply compression P-Box table to get 48 bit round key in binary.
- Append the round key to ***round_keys*** list and return *round_keys*.

Task 6: Input Handling

- Write code to accept a key in hexadecimal (16 hex digits) and convert it to 64-bit binary.
- Call ***generate_round_keys(key_64bit)*** method to get ***round_keys*** list.
- Display the ***round_keys*** that are in hexadecimal.

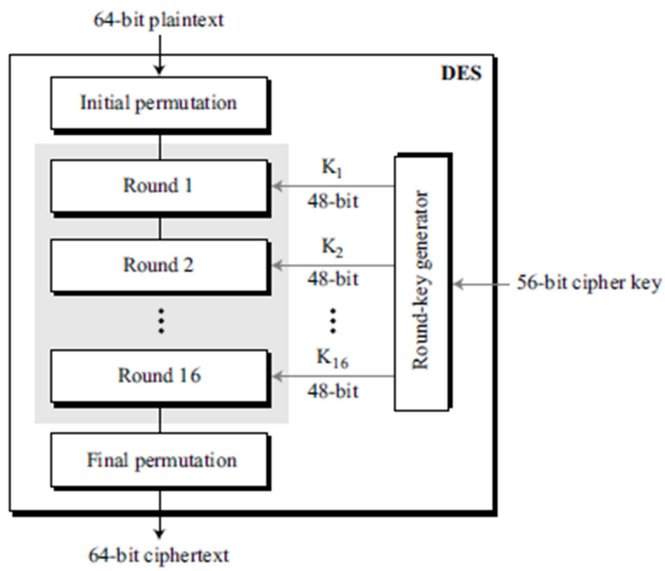
DES Encryption

****

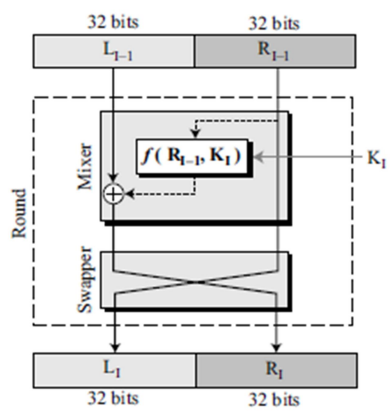
Objective-2: Write a Python program to implement DES encryption.

Task 0: Understand the DES encryption process

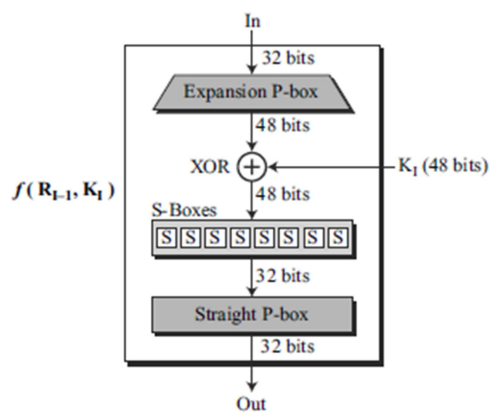
General structure of DES



A round in DES (encryption site)



DES function



Task 1: Define Required Tables

Create all necessary tables required for DES encryption:

- Initial Permutation Table (IP)
- Final Permutation Table (FP)
- Expansion P-Box Table (to expand 32-bit data to 48-bit)
- Straight P-Box Table (for permutation after S-box substitution)
- Eight S-Boxes (for substitution step)

Table 6.1 *Initial and final permutation tables*

Initial Permutation	*Final Permutation*
58 50 42 34 26 18 10 02	40 08 48 16 56 24 64 32
60 52 44 36 28 20 12 04	39 07 47 15 55 23 63 31
62 54 46 38 30 22 14 06	38 06 46 14 54 22 62 30
64 56 48 40 32 24 16 08	37 05 45 13 53 21 61 29
57 49 41 33 25 17 09 01	36 04 44 12 52 20 60 28
59 51 43 35 27 19 11 03	35 03 43 11 51 19 59 27
61 53 45 37 29 21 13 05	34 02 42 10 50 18 58 26
63 55 47 39 31 23 15 07	33 01 41 09 49 17 57 25

Figure 6.6 *Expansion permutation*

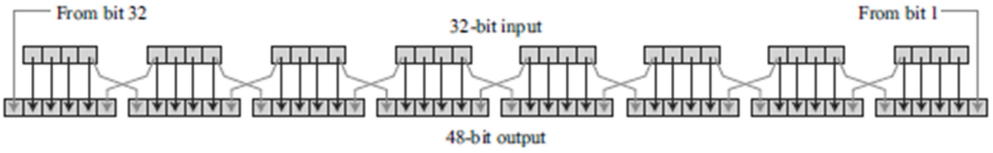


Table 6.2 *Expansion P-box table*

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	01

Table 6.11 *Straight permutation table*

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Table 6.3 *S-box 1*

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

Table 6.4 *S-box 2*

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	01	08	14	06	11	03	04	09	07	02	13	12	00	05	10
1	03	13	04	07	15	02	08	14	12	00	01	10	06	09	11	05
2	00	14	07	11	10	04	13	01	05	08	12	06	09	03	02	15
3	13	08	10	01	03	15	04	02	11	06	07	12	00	05	14	09

Table 6.5 *S-box 3*

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	00	09	14	06	03	15	05	01	13	12	07	11	04	02	08
1	13	07	00	09	03	04	06	10	02	08	05	14	12	11	15	01
2	13	06	04	09	08	15	03	00	11	01	02	12	05	10	14	07
3	01	10	13	00	06	09	08	07	04	15	14	03	11	05	02	12

Table 6.6 *S-box 4*

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	07	13	14	03	00	6	09	10	1	02	08	05	11	12	04	15
1	13	08	11	05	06	15	00	03	04	07	02	12	01	10	14	09
2	10	06	09	00	12	11	07	13	15	01	03	14	05	02	08	04
3	03	15	00	06	10	01	13	08	09	04	05	11	12	07	02	14

Table 6.7 *S-box 5*

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	02	12	04	01	07	10	11	06	08	05	03	15	13	00	14	09
1	14	11	02	12	04	07	13	01	05	00	15	10	03	09	08	06
2	04	02	01	11	10	13	07	08	15	09	12	05	06	03	00	14
3	11	08	12	07	01	14	02	13	06	15	00	09	10	04	05	03

Table 6.8 *S-box 6*

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	01	10	15	09	02	06	08	00	13	03	04	14	07	05	11
1	10	15	04	02	07	12	09	05	06	01	13	14	00	11	03	08
2	09	14	15	05	02	08	12	03	07	00	04	10	01	13	11	06
3	04	03	02	12	09	05	15	10	11	14	01	07	10	00	08	13

Table 6.9 *S-box 7*

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	4	11	2	14	15	00	08	13	03	12	09	07	05	10	06	01
1	13	00	11	07	04	09	01	10	14	03	05	12	02	15	08	06
2	01	04	11	13	12	03	07	14	10	15	06	08	00	05	09	02
3	06	11	13	08	01	04	10	07	09	05	00	15	14	02	03	12

Table 6.10 *S-box 8*

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	02	08	04	06	15	11	01	10	09	03	14	05	00	12	07
1	01	15	13	08	10	03	07	04	12	05	06	11	10	14	09	02
2	07	11	04	01	09	12	14	02	00	06	10	10	15	03	05	08
3	02	01	14	07	04	10	8	13	15	12	09	09	03	05	06	11

Task 2: Write XOR Function

Implement a function `xor(a, b)` to perform bitwise XOR operation between two binary strings of equal length. This is used in DES rounds for mixing data with round keys.

Task 3: Implement S-Box Substitution

Define a function `sbox_substitution(data)` to

- Divide the 48-bit input into eight 6-bit chunks each input to eight S-Boxes.
- Determine row using first and last bits.
- Determine column using middle four bits.
- Substitute values using S-Boxes.
- Convert result into 32-bit output.

Task 4: Implement DES Round Function (f-function)

Write a function `des_function(right, round_key)` to develop the DES function that operates on the right half:

- Expand 32-bit input to 48-bit using expansion table.
- Apply XOR with round key.
- Perform S-box substitution.
- Apply straight permutation.

Task 5: Implement DES Encryption

Develop a function `des_encrypt(plaintext, round_keys)` to implement the full DES encryption process:

- Apply initial permutation on plaintext.
- Split into left and right halves.
- Perform 16 rounds of processing:
  - Apply DES function
  - XOR with left half
  - Swap halves
- Combine halves and apply final permutation.

Task 6: Input Handling and Output

Accept plaintext input → Convert to binary → Perform encryption → Convert ciphertext to hexadecimal → Display ciphertext.